

프론트-에이전트 통합 테스트 보고서

LOVV V2 · 화면 측 × AI 측 계약 정합성 검증

문서명 LOVV V2 통합 테스트 결과서	문서 버전 v1.0 (최종)
작성일 2026-07-13	종합 판정 조건부 합격 (조건부 PASS)
배포 런타임 Bedrock AgentCore · us-east-1	근거 문서 화면 E2E + 에이전트 테스트

본 보고서는 LOVV V2 여행 일정 서비스에서 **화면(프론트엔드)**과 **AI(에이전트)**를 잇는 구간을, 화면 측(브라우저 E2E)과 AI 측(로컬·배포 실측) **양쪽에서 측정**해 하나의 계약(V2)이 양면에서 지켜지는지 검증한 결과를 기록한다.

목차

문서 개정 이력

- 무엇을, 왜 테스트했나 (목적·범위)
- 어떻게 나눠 테스트했나 (전략)
- 테스트 환경과 구성
- 테스트 방법

5. 결과 — 화면(프론트엔드)

6. 결과 — AI(에이전트)

7. 통합 정합성 — 화면과 AI가 같은 약속을 지키는가 (핵심)

8. 종합 판정

9. 부록 — 요청·응답 형식 예시

문서 개정 이력

버전	일자	주요 변경
v0.9	2026-07-13	초안 — 화면(E2E)·AI(로컬·배포) 양측 실측을 통합 계약 관점으로 결합
v1.0	2026-07-13	내용 보강 및 최종 검토 — 환경 요약, 커버리지 설계, 대표 테스트 케이스, 요청·응답 형식 예시 추가. 용어 평이화 및 문서 내 상호 정합성 정리

1. 무엇을, 왜 테스트했나 (목적·범위)

프론트엔드와 에이전트는 **V2 계약**이라는 한 지점에서 맞물린다. 화면이 규칙대로 요청을 만들어도 AI가 그 규칙을 안 지키면 서비스가 깨지고, 반대도 마찬가지다. 그래서 이 약속을 양쪽에서 확인하는 것이 이번 테스트의 목적이다.

- 화면 쪽 질문:** 클라이언트가 계약대로 요청을 만들고, 돌아온 응답을 제대로 화면에 그리고 저장·재조회하는가?
- AI 쪽 질문:** 실제 배포된 AI가 그 계약을 실제로 이행해 올바른 응답·상태 전환·예외 처리를 하는가?

검증 범위: 일정 생성 → 추가 질문 답변 → 일부 수정 → 저장 → 저장 목록 → 저장 상세 재진입 이라는 전체 흐름과, 그 과정에서 대화가 끊기지 않도록 유지되는 표식(세션·스레드), 그리고 응답에 담기는 정보(상항 코드·일정 항목 형식).

2. 어떻게 나눠 테스트했나 (전략)

통합 지점(계약)을 두 축으로 나눠 확인하고, 두 결과를 계약 항목별로 맞대어 봤다.

축	무엇을 증명하나	방식 (쉬운 설명)
화면 축 — E2E	화면이 규칙대로 요청을 만들고, 응답을 그리고·저장·재조회한다	브라우저 자동화(Playwright)로 실제 클릭·화면 흐름을 재현. 이때 서버는 약속된 가짜 응답(모킹) 으로 대체해, 화면 로직이 규칙을 지키는지만 흔들림 없이 확인
AI 축 — 로컬 + 배포	실제 AI가 그 규칙을 지켜 동작한다	개발 PC에서 코드를 직접 실행(로컬)하고, 실제 배포된 AI를 진짜로 호출(배포) 해 응답·상태 전환·예외를 확인

왜 화면은 가짜 응답을 썼나? 실제 AI 호출은 비용이 들고 응답 시간이 들쭉날쭉해, 화면 로직의 옳고 그름을 반복 검증하기엔 부적합하다. 그래서 화면 쪽은 **약속(계약)면**을 결정론적으로 검증하고, AI 쪽은 **실제 이행면**을 진짜 호출로 검증하는 역할 분담을 택했다. 로컬과 배포 시간 측정 은 서로 섞지 않고 따로 기록했다.

3. 테스트 환경과 구성

3.1 화면(프론트엔드)

- 도구: **Playwright 1.61.1**(브라우저 자동화 도구), Chromium 브라우저
- 테스트 파일: `Lovv-pg/frontend/e2e/lovv-v2-itinerary.e2e.ts`
- 방식: 화면-서버 사이 V2 요청·응답 규칙을 **가짜 응답(모킹)**으로 검증. 실행 중 실제 AWS(API Gateway·Lambda·AgentCore)나 지도·이미지·폰트 서버를 호출하지 않도록 외부 통신을 차단(격리)했다.

3.2 AI(에이전트)

- 위치(Region): `us-east-1`
- 사용 도구(쉬운 설명): **Bedrock**(AI 언어모델 호출), **벡터 검색**(S3 Vectors, 의미 기반 장소·도시 검색), **데이터베이스**(DynamoDB, 장소·축제 상세 조회), **경로 계산**(ORS, 이동 시간)
- 로컬: 개발 PC 안에서 코드를 그대로 실행. AI 기억(Memory) 기능은 비용·과거 오류 이력 때문에 꺼 두고 `( LOVV_MEMORY_ENABLED=false )` 실행
- 배포: 실제 올라간 AI를 **직접 호출**. 이때 "같은 작업을 이어간다"는 표식(세션·스레드·사용자 식별자)을 함께 넣어야 하며, 안 넣으면 AI가 대화 상태를 못 찾아 오류(500)를 낸다
- 모든 실제 호출은 각 **60초 제한**으로 실행
- 증빙 자료: 로컬 `docs/tasks/results/v2_agent_test_local/20260713T120000Z/`, 배포 `docs/tasks/results/v2_agent_test_deployed/20260713T120000Z/`

3.3 화면과 AI를 잇는 약속(V2 계약)

이 항목들이 곧 "통합"의 접점이며, 뒤에서 양쪽이 이걸 지키는지 확인한다.

- **요청 순서:** 생성 → 추가 질문 → 수정 → 저장 → 저장 목록 → 저장 상세
- **세션 이어가기:** 생성·추가 질문·수정이 같은 세션 표식을 사용 (같은 사용자의 이어지는 작업)
- **스레드 이어가기:** 추가 질문·수정이 생성 때 받은 같은 대화 표식을 재사용
- **현재 일정 전달:** 수정 요청에 "지금 화면의 일정과 순서"를 함께 보내, AI가 그 위에서 고침
- **상황 코드:** 날씨 위험 대체 제안 / 후보 부족 되물음 / 축제 없음 / 지원 밖 지역 등
- **일정 항목 형식:** 각 장소에 출처 종류, 실내·실외 구분, AI가 쓴 소개·추천 이유가 담김

3.4 환경 요약

항목	값
배포 런타임	AWS Bedrock AgentCore Runtime (관리형 서버리스 — 호출마다 격리 실행)
런타임 ARN	<code>arn:aws:bedrock-agentcore:us-east-1:925273580929:runtime/LovvAgentCore_LovvAgentV2-cy3tYk7nV4</code>
리전	<code>us-east-1</code>
코드 대상	<code>src/lovv_agent_v2/</code> (로컬 = 배포 동일 로직)
파이프라인	LangGraph 멀티 에이전트 (Intent · Supervisor · City Select · Planner · Explain 등)
AI·검색·DB·경로	Bedrock(언어모델·임베딩) · S3 Vectors(의미 검색) · DynamoDB(상세 조회) · ORS(경로)
로컬 실행	개발 PC in-process, AI 기억 off(<code>LOVV_MEMORY_ENABLED=false</code>)
배포 실행	AgentCore 직접 호출, 세션·스레드·사용자 식별자 주입(<code>runtimeSessionId</code> 33자 이상)
호출 상한	요청당 60초
화면 테스트	Playwright 1.61.1 · Chromium, 결정론적 API 모킹(외부 통신 차단)
관측 제약	배포 직접 호출 응답의 <code>traceId</code> 가 null → 응답 본문 + 구조화 <code>AGENT_*</code> 로그로 증빙

4. 테스트 방법

4.1 화면 — 11단계 사용자 흐름

로그인·초기 설정을 마친 사용자가 여행 플래너에서 일정을 만들고, 추가 질문에 답하고, 일부를 수정·저장한 뒤, 마이페이지에서 다시 여는 **전체 흐름을 하나의 시나리오로** 재현했다. 각 단계마다 "규칙을 지켰는지"를 자동으로 확인(assert)한다.

4.2 AI — 4개 경로

경로	목적 (쉬운 설명)	실행
경로 A — 규칙·회귀	코드를 바꾼 뒤에도 기본 규칙과 예전 동작이 그대로인지 자동 확인	<code>pytest</code> (파이썬 테스트)로 실행
경로 B — 로컬 실행	개발 PC에서 생성·수정·재개를 실제로 돌려 봄	코드 직접 실행, 기억 기능 끄

경로	목적 (쉬운 설명)	실행
경로 C — 배포 실행	실제 서비스에 올라간 AI를 진짜로 호출	60초 제한, 대표 사례 16개 + 반복 측정
이전 버전(V1)과 비교	같은 뜻의 요청을 옛 버전과 새 버전에 넣어 속도 비교	대표 4개(P01~P04) 각 1회

4.3 테스트 커버리지 — 왜 이 사례들인가

"많이"가 아니라 **에이전트가 실제로 갈라지는 경로를 한 번씩 덮는 최소 구성**을 목표로 사례를 골랐다.

- **생성 경로:** 단일/복수 테마, '조용함/활기' 혼잡 조건, 장소 직접 지정(anchor), 후보가 얇은 희소 테마(온천·휴양), 축제 포함/미포함 — 도시 선택·일정 생성이 조합마다 다른 경로를 타므로 대표 조합을 고루 포함.
- **수정 경로:** 한 장소 교체(검색 유무), 대표(Seed) 장소 교체, 같은 날/다른 날 여러 장소 교체, 하루 전체 재구성, 명시/자동 도시 변경 — 부분 수정의 각 분기를 개별 검증.
- **되물음·재개 경로:** 날씨 위험 대체, 후보 부족 조건 완화, 축제 없음 이어가기 — 멈췄다 이어가는 상태 유지를 검증.
- **예외·경계:** 지원하지 않는 지역 입력 — 안전 거부·경계 처리를 검증.

화면 쪽은 이 계약을 관통하는 대표 흐름 하나를 11단계로 세분해, 요청 형성부터 저장·재조회까지 한 번에 덮었다.

4.4 검증 기준 (합격 조건)

검증 항목	합격 조건
요청 순서	생성→추가질문→수정→저장→목록→상세가 설계 순서대로 전송·처리
세션·스레드 연속성	이어지는 요청이 같은 세션·대화 표식 유지, 재개가 이전 결과 위에서 이뤄짐
현재 일정 전달	수정 요청의 현재 일정·순서를 AI가 복원해 그 위에서 교체
상황 코드	날씨·후보 부족·축제 없음·지원 밖을 정직하게 코드로 반환·재개
일정 항목 형식	응답 항목에 출처·실내외·소개·이유 필드 존재
예외 처리	잘못된/지원 밖 입력을 오류 전파 없이 안전 처리
안정성	정상 시나리오 오류 0, 무한 반복·시간초과 0

5. 결과 — 화면(프론트엔드)

5.1 단계별 결과 — 11단계 전부 통과

순서	사용자 행동	확인한 내용	결과
1	로그인 후 플래너 진입	로그인 상태·초기 설정 적용, 플래너 화면 정상 표시	통과
2	동해시 1박 2일 만들기 요청	요청에 목적지·기간·요청 유형·세션 표식이 올바르게 담김	통과
3	추가 질문 선택지 확인	응답의 질문·선택지·선택지별 도움말이 화면에 제대로 표시	통과
4	추가 질문에 답변	답변이 만들기 다음 순서로 전송되고, 같은 세션·대화로 이어짐	통과

순서	사용자 행동	확인한 내용	결과
5	만들어진 일정 확인	1일차 일정과 첫 장소(목호등대)가 화면에 표시	통과
6	일정 수정 요청	"1일차 첫 장소 바뀌줘..." 요청에 현재 순서·세션·대화 연속성이 담김	통과
7	수정 결과 확인	1일차 첫 장소가 목호등대 → 한섬감성바닷길로 바뀌고, 이전 장소는 사라짐	통과
8	일정 저장	저장 중 버튼 잠김, 저장 요청에 제목·목적지·기간·테마·선호 조건·수정된 일정이 포함	통과
9	마이페이지 저장 목록	저장 1건, 제목·요약이 일치	통과
10	저장 상세 다시 열기	주소·목적지·기간·수정한 장소가 유지되고, 지운 장소가 다시 나타나지 않음	통과
11	잘못된 요청 차단	허용되지 않은 통신을 테스트가 막고 기록	통과

5.2 약속(계약)을 지켰는가 — 확인 결과

- 요청 순서(생성 → 추가 질문 → 수정 → 저장 → 목록 → 상세)를 그대로 따름
- 세션 이어가기: 생성·추가 질문·수정이 같은 세션 표식 사용
- 스레드 이어가기: 추가 질문·수정이 생성 때 받은 같은 대화 표식 사용
- 현재 일정 전달: 수정 요청에 지금 화면의 일정·순서가 담김
- 저장 데이터: 최종 수정 일정·선호 조건·목적지·기간·테마가 모두 포함
- 재조회 일관성: 저장 목록·상세에서 최종 수정 결과가 유지됨
- 외부 통신 차단: 실제 AWS·지도·이미지·폰트 호출 없이 실행

5.3 실행이 안정적인가 — 반복·회귀

- 브라우저 자동화 테스트: 따로 실행 1회 통과, 전체 실행 1회 통과, 보강 실행 1회 통과 — **서로 다른 브라우저 실행에서 반복 통과**해 우연한 성공이 아님을 확인
- 회귀(예전 동작 유지): **자동 테스트 29개 파일·323개 항목 전부 통과**, 코드 스타일 검사 통과, 빌드(배포용 묶기) 통과

6. 결과 — AI(에이전트)

6.1 경로별 요약

구분	실행	통과	시간초과	비고 (쉬운 설명)
경로 A — 규칙·회귀	46	46	0	기본 규칙·해석·되물음·배포 호출기 자동 테스트
경로 B — 로컬 실행	36	35	0	실패 1건은 '도시 변경' 사례를 단독 으로 돌렸을 때. 이건 앞선 생성 결과가 있어야 하는 요청이라, 생성→도시변경을 이어서 돌린 경우엔 정상 통과
경로 C — 배포 실행	16	15	0	15건 정상 응답(HTTP 200), 1건은 '지원하지 않는 지역' 요청이 응답 형식 목록에 없어 오류(500)

구분	실 행	통 과	시간초 과	비고 (쉬운 설명)
이전 버전(V1)과 비교	8	8	0	대표 4개를 옛/새 버전 각 1회

로컬에서 한 요청의 최대 소요는 47.406초, 배포에서 정상 응답의 최대는 25.334초였다. 재개(이어가기) 3개 사례 모두 기억 기능이 꺼진 상태로 정상 기록됐고, 무한 반복·시간초과·기억 관련 오류는 없었다.

6.2 로컬 실행 – 만들기·수정 동작 확인

단독 실행 30회(만들기 15 · 수정 15). 만들기 평균 22.933초, 수정 평균 14.537초, 최대 47.406초.

표 읽는 법: *slot replace* = 한 장소만 교체 · *seed replace* = 사용자가 콕 집어 넣은 대표 장소 교체 · *day regenerate* = 하루 전체를 다시 짜기 · *sequential fold* = 여러 곳을 순서대로 하나씩 반영 · *clarification* = 후보가 부족할 때 사용자에게 되묻는 것.

사례	결과	시간	확인한 것 (쉬운 설명)
역사·전통 만들기	영덕군, 2일 6곳	27.197초	도시 선택 → 일정 생성 → 설명까지 정상
자연+해안 2박3일	동해시, 3일 10곳	24.510초	여러 테마 일정 + 날씨 대체 질문
장소 직접 지정 만들기	동해시, 2일 5곳	18.513초	지정한 장소 기준으로 일정 생성
후보 적은 힐링 요청	1일 1곳 + 되물음	20.672초	후보가 부족하면 감추지 않고 되물음
한 장소만 교체(질문 없음)	영덕군, 2일 6곳	10.400초	부분 수정 경로 정상
대표 장소 교체	2일 5곳	8.340초	고정 장소 교체 규칙 정상
같은 날 여러 곳 수정	영덕군, 2일 6곳	11.549초	여러 곳 순차 반영 정상
하루 전체 다시 짜기	동해시, 3일 10곳	10.810초	해당 날짜만 재구성

6.3 되물음 → 이어가기(재개) 사례

사용자에게 한 번 되물은 뒤, 답을 받아 **이전 결과 위에서** 이어가는 흐름이다.

사례	처음 만들기	이어가기	결과 (쉬운 설명)
날씨 대체	35.991초 · '날씨 위험, 대체 제안'	8.978초	같은 세션에서 대체 일정으로 정상 이어짐
후보 부족 → 조건 완화	19.789초 · '후보 부족'	13.319초	테마를 완화한 뒤 4곳 일정 생성
도시 바꾸기	27.525초 · 동해시	13.080초 · 강릉시	도시를 바꿔도 10곳 항목이 그대로 유지

6.4 배포 실행 – 대표 16개 사례

실제 배포된 AI를 직접 호출한 결과(HTTP 200 = 정상 응답).

번호	결과	시간	확인한 것 (쉬운 설명)
1 기본 만들기	200	21.427초	영덕군 2일

번호	결과	시간	확인한 것 (쉬운 설명)
2 여러 테마	200	15.867초	동해시 3일, 날씨 안내
3 장소 직접 지정	200	12.406초	경주시 1일, 지정 장소 기반
4 축제 포함	200	17.955초	확정 축제가 있는 도시·달 경로
5 축제 없음	200	3.080초	'그 달 축제 없음' 되물음
6 후보 적은 힐링	200	16.700초	제한 상황을 정직하게 표시
7 날씨 대체 제안	200	19.131초	'날씨 위험, 대체 제안'
8 날씨 대체 이어가기	200	5.679초	같은 대화 이어서 재개
9 한 장소 교체(질문 없음)	200	8.660초	여분 후보로 빠르게 교체
10 한 장소 교체(검색 포함)	200	8.510초	추가 검색 + 설명 생성
11 여러 장소 교체	200	14.090초	순차 반영
12 하루 다시 짜기	200	17.154초	해당 날짜만 재구성
13 도시 바꾸기(지정)	200	12.949초	동해시 입력을 경주시로 다시 고름
14 도시 바꾸기(자동)	200	24.547초	동해시를 빼고 삼척시로 새로 찾음
15 축제 없음 이어가기	200	12.618초	'축제 없이 계속' 적용
16 지원 밖 지역	500	0.998초	'지원하지 않는 지역'이 응답 형식 목록에 없어 오류 → 코드에 목록 추가 반영

반복 측정(같은 요청을 여러 번 돌려 안정성 확인)

흐름	반복	평균	중앙값	최소~최대
기본 만들기	5	16.339초	15.405초	11.493~20.785초
한 장소 교체(질문 없음)	5	9.846초	8.732초	3.970~16.285초

참고: 이번 배포 호출은 상세 추적 표식(traceId)이 비어 있어, 결과 증빙은 **응답 본문**과 **구조화된 실행 로그**를 1차 근거로 사용했다.

6.5 실제로 어떤 도구를 썼나 (관측)

로컬 실행 로그에서 아래 도구 호출을 실제로 확인했다 — 즉 AI가 말로만 답한 게 아니라 진짜 검색·계산을 거쳤다는 증거다.

- **AI 언어모델 호출(Bedrock)**: 요청 해석·설명 문구 생성
- **의미 검색(벡터)**: 도시·장소 후보 찾기
- **데이터베이스 조회**: 장소·축제 상세 채우기
- **경로 계산(ORS)**: 이동 가능성 확인

흐름	내부 처리시간	의미 검색	DB 조회	AI 모델 호출	경로 계산
단일 해안	7.446초	3회 / 261ms	2회 / 53ms	2회 / 4.146초	1회 / 481ms
자연+해안	8.883초	4회 / 331ms	2회 / 58ms	2회 / 5.553초	1회 / 424ms
장소 직접 지정	7.093초	1회 / 106ms	1회 / 43ms	2회 / 4.155초	1회 / 435ms
축제 포함	17.031초	4회 / 282ms	2회 / 22ms + 1회 / 180ms	2회 / 13.977초	1회 / 432ms

정리하면 — 의미 검색은 100~330ms, DB 조회는 20~60ms로 **1초를 한참 밑돌고**, 시간은 사실상 전부 **AI 언어모델 호출(4~14초)**에 쏠린다. 즉 최적화 대상은 검색·DB가 아니라 LLM 호출이다.

6.6 이전 버전(V1)과 속도 비교

같은 뜻의 요청을 옛 버전과 새 버전에 각각 넣어 전체 소요 시간을 비교했다.

흐름	V1(옛)	V2(새)	해석
단일 해안	22.949초	15.131초	V2가 7.818초 빠름
자연+해안	37.634초	16.040초	V2가 21.594초 빠름
장소 직접 지정	12.730초	14.294초	V2가 1.564초 느림
축제 포함	21.613초	23.700초	V2가 2.087초 느림

새 버전은 의미 검색을 **병렬로**, 상세 조회를 **한 번에 묶어**(옛 버전은 하나씩 여러 번) 처리하도록 바꿨다. 그래서 검색·DB 왕복이 줄었고, 남은 최적화 여지는 AI 모델 호출 쪽이다.

6.7 대표 테스트 케이스 (정식)

각 경로의 대표 케이스를 **전제 → 입력 → 실행 단계 → 기대 → 실측 → 판정** 양식으로 기술한다.

TC-A — 생성 + 축제 (핵심 복합 경로 · 배포 실행)

구분	내용
전제	확정 축제가 있는 도시·달이 데이터에 적재, 배포 런타임 직접 호출
입력	축제 포함 생성 요청(테마 + 축제 포함 + 여행 월), 세션·스레드 포식 주입
실행 단계	① Intent 요청 정리 → ② Festival 확정 축제 판별 → ③ City Select 도시 선택 → ④ Planner 일정 생성·축제 배치 → ⑤ 응답
기대	HTTP 200, 확정 축제 포함 일정, 날씨 대체 제안
실측	HTTP 200 · 17.955초 , 축제 포함 city/month 경로, 날씨 대체 제안 동반
판정	Pass — 축제 판별·배치가 한 요청에서 정상

TC-B — 되돌음 → 이어가기 (상태 유지 · 로컬 실행)

구분	내용
전제	야외 비중이 높은 일정에서 날씨 위험이 감지됨

구분	내용
입력	① 생성 요청 → '대체 볼까요?' 되물음 → ② 같은 세션에서 '대체 보기' 선택으로 재개
실행 단계	생성 → <code>weather_alternative_available</code> 반환(대기) → 사용자 선택 → 중단 지점부터 재개 → 실내 위주 대체 일정
기대	같은 세션에서 대체 일정으로 정상 재개, 무한 반복·오류 0
실측	생성 35.991초 (<code>weather_alternative_available</code>) → 재개 8.978초 , 동일 session 대체 일정 정상. 기억 off 기록, 무한 반복·시간초과 0
판정	Pass — 되물음 후 상태를 잃지 않고 이어감

TC-C — 명시 도시 변경 (별도 경로 + 상태 유지 · 로컬 실행)

구분	내용
전제	생성으로 동해시 일정이 만들어진 상태(선행 상태 필요)
입력	같은 세션에서 "강릉시로 바꿔줘"
실행 단계	도시 변경은 부분 수정이 아니라 City Select 부터 다시 도는 별도 경로 로 처리
기대	도시 식별자 교체, 항목 수 유지
실측	생성 27.525초 (동해시) → 변경 13.080초 (강릉시), <code>KR-51-170</code> → <code>KR-51-150</code> , 10개 항목 유지
판정	Pass — 별도 경로로 정상, 상태 이어감
비고	이 요청을 단독 (선행 생성 없이)으로 돌리면 선행 상태가 없어 실패 — 요청 특성상 정상 (§ 6.1 참조)

TC-D — 지원 밖 지역 (경계·예외 · 배포 실행)

구분	내용
전제	지원하지 않는 지역을 요청
입력	지원 밖 지역 입력, 배포 직접 호출
실행 단계	경계 감지 → 예외 처리 응답
기대	오류 전파 없이 안전한 경계 처리
실측	HTTP 500 · 0.998초 — <code>unsupported_region</code> 코드가 응답 형식 허용 목록에 없어 실패. 코드에 허용 목록 추가 반영 , 규칙 회귀 통과, 다음 배포에서 재확인 예정
판정	조건부 — 경계는 감지했으나 응답 형식 계약 불일치로 500. 소스 보완 완료, 배포 재확인 필요

7. 통합 정합성 — 화면과 AI가 같은 약속을 지키는가 (핵심)

화면이 만든 약속을, 실제 배포된 AI가 똑같이 이행하는지 **항목별로 맞대어** 확인했다.

약속(계약) 항목	화면 쪽에서 확인	AI 쪽에서 확인	일치
요청 순서(생성→추가질문→수정→저장)	11단계 흐름에서 순서·조건 통과	로컬·배포에서 생성→수정·재개 이어짐	✓
세션 이어가기	세 요청이 같은 세션 표식 사용	세션 표식을 넣자 같은 작업으로 재개	✓
스레드(대화) 이어가기	생성 때 받은 대화 표식을 재사용	재개 사례가 같은 대화로 이어짐	✓
현재 일정 전달	수정 요청에 현재 일정·순서 포함	전달된 상태를 복원해 그 위에서 교체	✓
추가 질문·상황 코드	선택지·도움말 정상 표시	날씨 대체·후보 부족·촉제 없음 코드 반환·재개	✓
일정 항목 형식	저장·재조회에서 수정 결과 유지	출처·실내외·소개·이유가 응답에 담김	✓
예외·경계 처리	잘못된 요청 차단·기록	'지원 밖 지역' 경계 처리(목록 반영)	✓

요약: 7개 약속 항목 모두, 화면이 기대한 것과 AI가 실제로 한 것이 일치했다.

8. 종합 판정

- **화면:** 핵심 흐름(만들기·추가 질문·수정·저장·목록·상세 재진입) 11단계 전부 통과, 약속·세션·대화 연속성 일치, 회귀 323건 통과.
- **AI:** 규칙·회귀 46/46, 로컬 실행 36/35(시간초과 0), 배포 실행 16/15(정상 응답), 이전 버전 비교 8/8. 실제 AI 호출·의미 검색·DB 조회·경로 계산까지 관측.
- **통합:** 화면이 만든 V2 약속을 실제 배포 AI가 똑같이 이행함을 7개 항목에서 확인했다. **통합 약속 정합성 조건부 합격** — 주요 경로는 일치했고, '지원 밖 지역' 1건만 다음 배포에서 재확인 예정이다.

관측된 개별 사항(감추지 않음): 로컬 '도시 변경' 단독 실패는 AI 오류가 아니라 "앞선 생성 결과가 있어야 하는 요청"이라는 특성 때문이고(이어서 돌리면 정상), 16번 '지원 밖 지역' 오류는 응답 형식 목록에 해당 코드를 추가 반영했다.

9. 부록 — 요청·응답 형식 예시

주의: 아래는 캡처된 실측 dump가 **아니라**, 문서에 기재된 필드와 실제 사례값으로 **재구성한 형식 예시(스키마)**다. 요청·응답에 계약 항목이 어떻게 담기는지 보여주기 위한 것이며, 실제 필드명·구조는 구현에 따라 다를 수 있다. 실제 캡처 원본은 § 3.2의 증빙 자료 경로에 있다.

① 생성 요청 (화면 → AI)

```
{
  "requestType": "create",
  "entryType": "chat",
  "destination": "동해시",
  "tripDuration": "1박 2일",
  "themes": ["sea_coast"],
  "includeFestivals": true,
  "sessionId": "<고정 세션 표식>"
}
```

② 추가 질문 응답 (AI → 화면) — 상황 코드 동반

```
{
  "responseStatus": "clarification",
  "reasonCode": "weather_alternative_available",
  "question": "이 시기 야외 일정에 날씨 위험이 있어요. 대체 일정을 볼까요?",
  "options": [
    { "optionId": "see_alternative", "label": "대체 일정 보기", "helperText": "실내 위주로 다시 구성합니다" },
    { "optionId": "keep_current", "label": "지금 일정 유지", "helperText": "현재 일정을 그대로 사용합니다" }
  ],
  "sessionId": "<고정 세션 표식>",
  "threadId": "<생성 응답에서 받은 대화 표식>"
}
```

③ 수정 요청 (화면 → AI) — 현재 일정 등보

```
{
  "requestType": "modify",
  "message": "1일차 첫 장소 바꿔줘. 더 조용한 곳을 원해.",
  "currentItinerary": {
    "day1": [
      { "order": 1, "placeName": "목호등대" },
      { "order": 2, "placeName": "..." }
    ]
  },
  "sessionId": "<고정 세션 표식>",
  "threadId": "<생성 응답에서 받은 대화 표식>"
}
```

④ 수정 응답 항목 (AI → 화면) — 교체된 항목의 형식

```
{
  "responseStatus": "completed",
  "changedItem": {
    "day": 1,
    "order": 1,
    "placeName": "한섬감성바닷길",
    "sourceType": "reserve_pool",
    "indoorOutdoor": "outdoor",
    "body": "한섬해변을 따라 걷는 조용한 산책로로, 붐빔이 적어 여유로운 아침에 어울립니다.",
    "reason": "더 조용한 곳을 원하셔서, 붐빔이 덜한 장소로 교체했습니다."
  },
  "unchangedPreserved": true
}
```

각 예시에서 확인되는 계약 항목: 세션·스레드 표식(①~③), 상황 코드(②), 현재 일정 전달(③), 일정 항목 형식(④:

sourceType · indoorOutdoor · body · reason), 나머지 보존(④: unchangedPreserved)